Machine Learning for Artificial Intelligence Engineering

Assignment 2
Course Project

Students:
Amer Delić (01331672)

Lecturers:
Umair Khan
Nicolas Pfob

Institute of Machine Learning and Neural Computation
Graz University of Technology

Graz, January 2026

Contents

1 Exploratory Data Analysis **3**

2 Baseline Model **7**

3 Model Experimentation and Validation **8**
3.1 Validation protocol and metrics 8
3.2 Experiment overview 8
3.3 Results by model family 8
3.3.1 Linear model: Logistic Regression 8
3.3.2 Kernel model: SVM 9
3.3.3 Tree ensembles: Random Forest, Gradient Boosting, XGBoost 10
3.3.4 Neural network: MLP 12
3.4 What improved performance and what did not 13
3.5 Best model and confusion matrix 13
3.6 If starting again from scratch 14

4 Outlier Detection **15**
4.1 Gaussian Mixture Model 15
4.2 Local Outlier Factor 15
4.3 Isolation Forest 15

Appendix **17**

List of Figures

1.1 Distribution of class labels 4
1.2 Feature boxplots 4
1.3 Feature correlation heatmap 5
1.4 PCA projection 5
1.5 PCA projection - Inlier vs Outlier 6
1.6 Feature distribution - inlier vs outlier 6

3.1 Confusion matrix of the final Random Forest model on a held-out inlier-only validation fold 14

1 Exploratory Data Analysis

To gain initial insight into the dataset and support the modeling process, a series of exploratory plots were generated. These are presented in Figure 1.1 - Figure 1.6 and highlight key aspects such as class distribution, feature correlations, outlier behavior, and geometric structure in reduced dimensions.
Figure 1.1 shows the distribution of class labels, revealing a moderate imbalance in which class 1 is strongly dominant and occurs considerably more frequently than the remaining classes, which appear at comparable levels. This imbalance justifies the use of the F1 score instead of accuracy during model evaluation to ensure fair performance across all classes and should also be considered when selecting an appropriate validation strategy. Figure 1.2 shows boxplots for all twelve features ($x_1 - x_{12}$), illustrating their value ranges, central tendencies, and outlier behavior. The plots reveal noticeable differences in scale across features, with several spanning broader ranges than others. All of the features exhibit slightly skewed distributions, while some of them contain extreme values outside the interquartile range, indicating the presence of outliers. These findings highlight the importance of feature scaling prior to model training, especially for distance-sensitive algorithms such as KNN or SVM. Figure 1.3 shows the feature correlation heatmap, revealing relatively high correlations across all features. This indicates strong redundancy in the dataset, suggesting that many features capture overlapping information. Such structure motivates the use of dimensionality reduction techniques like PCA to compress the feature space and improve model stability. Figure 1.4 shows the PCA projection of the dataset onto the first two principal components, with data points colored by class label. The plot reveals clear structure along the z_1 axis, forming three visibly distinct clusters. However, there is significant overlap between classes within these clusters, indicating that simple linear models will not be able to separate the classes effectively and that more flexible models, such as Random Forests or Neural Networks, may be required to capture the underlying structure. To investigate the outliers and their behaviour, two additional plots were created - a PCA projection and a feature distribution (KDE) comparison between inliers and outliers. Figure 1.5 shows the PCA projection with inliers and outliers labeled. As observed, outliers are not spatially isolated but instead embedded within the inlier clusters. This indicates that the outliers do not deviate strongly in terms of the global variance captured by PCA and may differ only through more subtle, localized patterns. Figure 1.6 shows the comparison of inlier and outlier feature distributions. Inlier distributions show sharper density curves with distinct peaks, reflecting the larger sample size, while outlier distributions appear smoother and flatter, which is expected due to the lower sample size. Despite these differences, the overall distributions are similar, which confirms what was already observed in Figure 1.5 and indicates that outliers may not be easily distinguishable through marginal analysis alone, highlighting the need for more complex outlier detection methods.

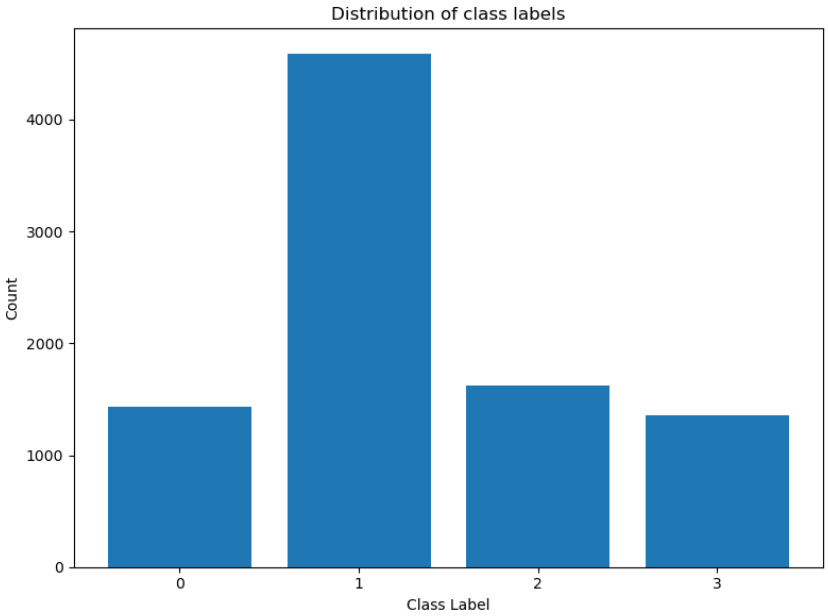

Figure 1.1: Distribution of class labels

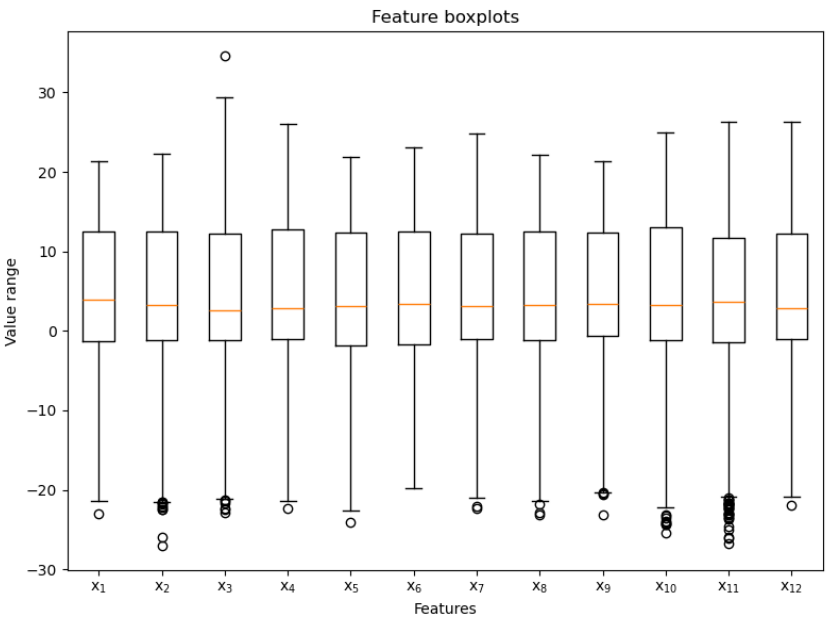

Figure 1.2: Feature boxplots

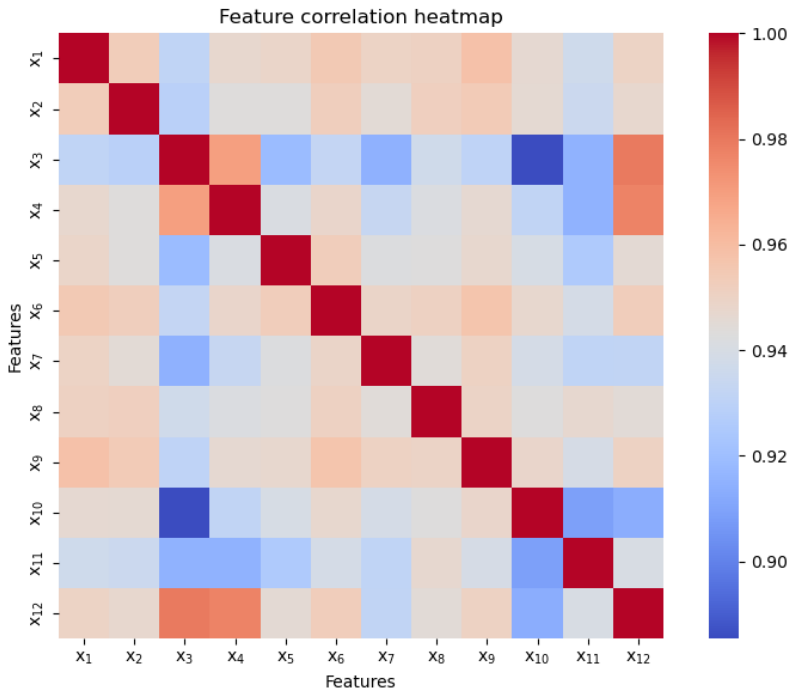

Figure 1.3: Feature correlation heatmap

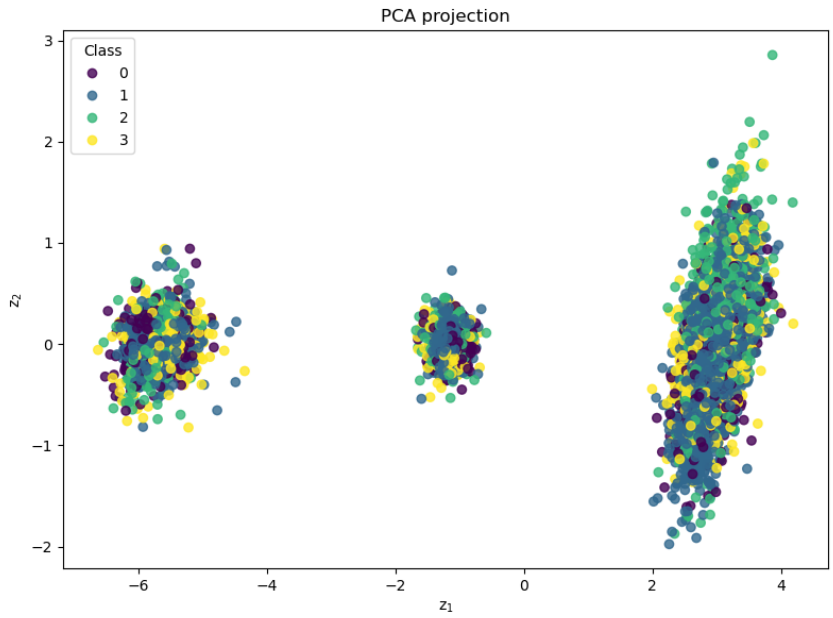

Figure 1.4: PCA projection

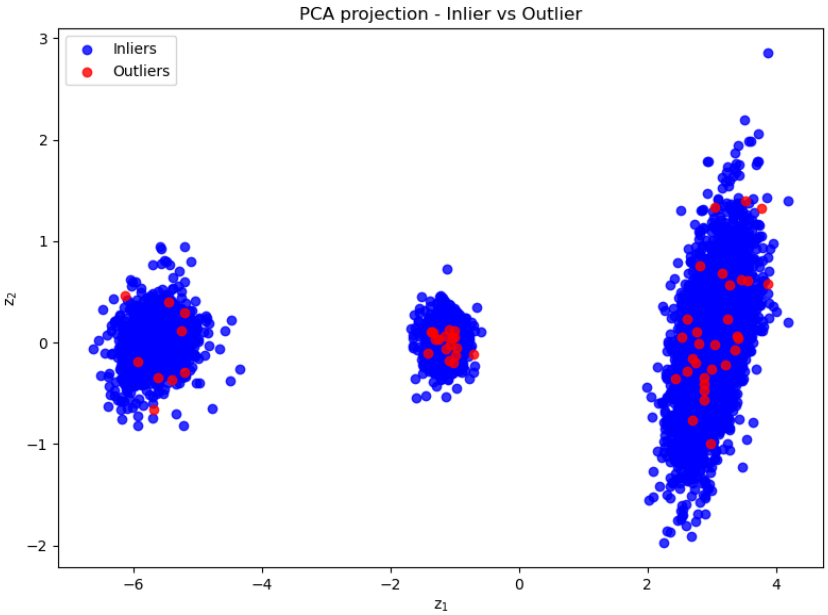

Figure 1.5: PCA projection - Inlier vs Outlier

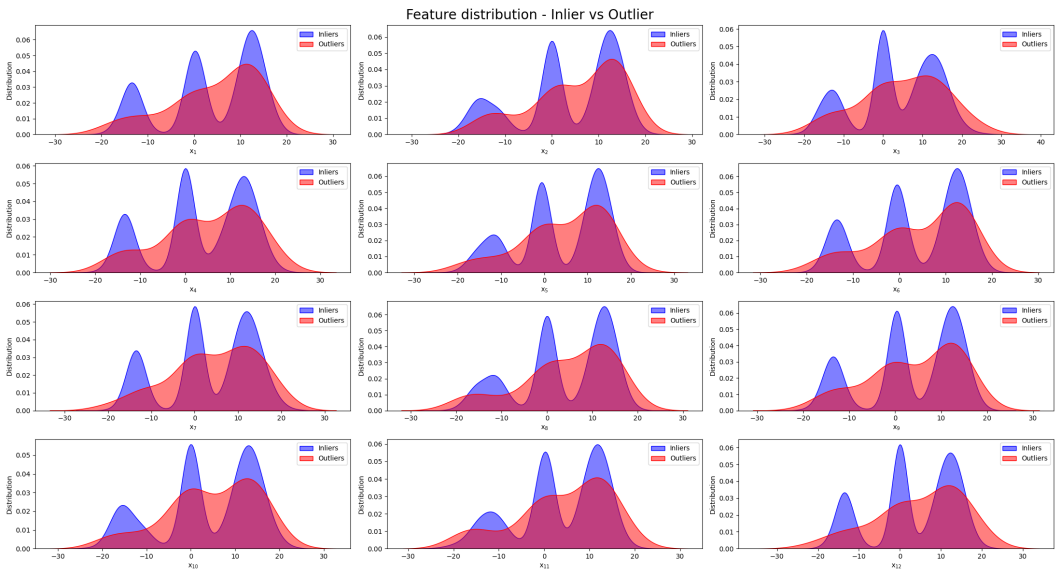

Figure 1.6: Feature distribution - inlier vs outlier

2 Baseline Model

Due to the size of the given dataset (9000 samples) and to ensure stable and reliable model evaluation, `K-Fold cross-validation` was selected as the validation strategy. Furthermore, due to the significant class imbalance it was decided to use the `Stratified` variant of K-Fold cross-validation. This approach maintains the original class distribution across all folds, which reduces the risk of biased performance estimates and ensures that each fold reflects the true diversity of the data. This is particularly relevant when computing per-class metrics such as *Precision*, *Recall* and *F1 score*, as it prevents minority classes from being underrepresented or missing in certain folds.

Before training the baseline model, feature scaling was applied independently within each fold using the `StandardScaler` from the *scikit-learn* package, ensuring that no information from the validation split leaked into the training process. As the baseline model, a simple `KNeighborsClassifier` (KNN) from the *sklearn* library was used. To determine an optimal value for the hyperparameter K, multiple KNN models with different K values were trained and evaluated using the validation strategy described above. Model performance was compared using the macro-averaged F1 score. The results are summarized in Table 2.1.

K	F1 score (macro averaged)
1	0.5075
3	0.5260
5	0.5429
7	0.5463
9	**0.5514**
11	0.5478
15	0.5441
21	0.5444

Table 2.1: Comparison of different KNN models

As can be observed, the KNN model with $K = 9$ achieves the highest macro-averaged F1 score and was therefore selected as the baseline model. The per-class metrics of the baseline model are summarized in Table 2.2.

Class	Precision	Recall	F1 score	Binary accuracy
0	0.6177	0.4410	0.5143	0.8674
1	0.6755	0.8792	0.7639	0.7231
2	0.5682	0.4898	0.5250	0.8401
3	0.6540	0.2909	0.4015	0.8700

Table 2.2: Baseline model metrics

3 Model Experimentation and Validation

3.1 Validation protocol and metrics

All models were evaluated using the same local validation protocol as the baseline stage to obtain a reliable estimate of generalization performance and to avoid common pitfalls such as preprocessing leakage.

We used stratified K-fold cross-validation, ensuring that class proportions are preserved across folds. Preprocessing (e.g., feature scaling where applicable) was fitted on the training split of each fold and applied to the corresponding validation split only.

For each experiment we report (i) per-class Precision, Recall, and F1-score for classes $y \in \{0, 1, 2, 3\}$, (ii) overall Accuracy, and (iii) the **macro-averaged F1-score** (unweighted mean of per-class F1). We use macro F1 as the main selection criterion because it weights all classes equally, preventing performance on frequent classes from dominating the assessment.

3.2 Experiment overview

We conducted substantially more than the required ten experiments and evaluated at least three distinct model families. The tested architectures and the number of hyperparameter configurations were:

- **Logistic Regression**: 3 configurations (Config 1–3)
- **SVM**: 3 configurations (Config 1–3)
- **Random Forest**: 3 configurations (Config 1–3)
- **Gradient Boosting**: 3 configurations (Config 1–3)
- **XGBoost**: 5 configurations (Config 1–5)
- **MLP**: 6 configurations (Config 1–6)

Across these experiments we iteratively adjusted capacity and regularization (e.g., regularization strength for Logistic Regression and SVM, tree ensemble complexity, and neural network architecture) and compared models using the same validation protocol and metrics.

3.3 Results by model family

3.3.1 Linear model: Logistic Regression

Logistic Regression performed poorly on this task, achieving only **macro F1** ≈ 0.246–0.259. We varied the regularization strength C (smaller C = stronger regularization) to test whether stronger constraints improve generalization, but the model remained limited by the linear decision boundary.

Config	C	Penalty	Solver
1	0.1	l2	lbfgs
2	1.0	l2	lbfgs
3	10.0	l2	lbfgs

Table 3.1: Logistic Regression hyperparameter configurations (all with max_iter = 2000).

The per-class recall reveals that the model effectively collapses to predicting a dominant class pattern (very high recall for class 1 but extremely low recall for classes 0 and 3). This indicates that the decision boundaries are not well captured by a linear model and that non-linear interactions among features are important.

Configuration	Class	Precision	Recall	F1	Binary accuracy	Accuracy	Macro F1
1	0	0.3697	0.0230	0.0429	0.8373	0.5221	0.2462
	1	0.5307	0.9610	0.6838	0.5473		
	2	0.4705	0.1422	0.2174	0.8162		
	3	0.2615	0.0221	0.0406	0.8433		
2	0	0.4052	0.0363	0.0661	0.8374	0.5226	0.2584
	1	0.5341	0.9509	0.6840	0.5526		
	2	0.4545	0.1551	0.2307	0.8137		
	3	0.2654	0.0294	0.0528	0.8414		
3	0	0.4052	0.0370	0.0672	0.8371	0.5220	**0.2588**
	1	0.5341	0.9492	0.6835	0.5523		
	2	0.4522	0.1564	0.2317	0.8132		
	3	0.2627	0.0294	0.0528	0.8413		

Table 3.2: Logistic Regression performance across tested configurations

3.3.2 Kernel model: SVM

SVM improves substantially over Logistic Regression but remains below the best tree ensembles.
We varied the penalty parameter C to control the margin/regularization trade-off while keeping an RBF kernel (with $\gamma = \texttt{scale}$) to introduce non-linearity.

Config	C	Kernel	γ
1	1.0	rbf	scale
2	3.0	rbf	scale
3	10.0	rbf	scale

Table 3.3: SVM hyperparameter configurations.

The best SVM configuration is **Config 3** with **macro F1 = 0.4559** and **Accuracy = 0.6058**. Similar to the linear model, SVM achieves strong recall for class 1 while other classes remain harder to capture, suggesting that additional model flexibility and/or robustness to atypical samples is needed.

Configuration	Class	Precision	Recall	F1	Binary accuracy	Accuracy	Macro F1
1	0	0.6291	0.1599	0.2534	0.8504	0.5748	0.3882
	1	0.5816	0.9452	0.7201	0.6257		
	2	0.5542	0.2289	0.3233	0.8274		
	3	0.4720	0.1760	0.2560	0.8460		
2	0	0.5558	0.2687	0.3617	0.8492	0.5920	0.4278
	1	0.6005	0.9348	0.7312	0.6499		
	2	0.5920	0.2627	0.3623	0.8334		
	3	0.5223	0.1701	0.2561	0.8514		
3	0	0.5699	0.3168	0.4065	0.8531	0.6058	**0.4559**
	1	0.6132	0.9298	0.7390	0.6654		
	2	0.5897	0.2935	0.3908	0.8350		
	3	0.5899	0.1907	0.2873	0.8580		

Table 3.4: SVM performance across tested configurations

3.3.3 Tree ensembles: Random Forest, Gradient Boosting, XGBoost

Tree-based ensembles consistently performed best.
For **Random Forest**, we increased the number of trees and tested a depth constraint to trade off variance reduction versus overfitting.

Config	n_estimators	max_depth	max_features
1	300	None	sqrt
2	600	None	sqrt
3	600	12	sqrt

Table 3.5: Random Forest hyperparameter configurations.

Random Forest achieved the strongest macro F1 among the supervised experiments prior to explicit outlier filtering. The best configuration is **RF Config 2** with **macro F1 = 4** and **Accuracy = 0.6937**. Compared to the linear and kernel models, Random Forest yields a more balanced per-class F1 profile.

Configuration	Class	Precision	Recall	F1	Binary accuracy	Accuracy	Macro F1
1	0	0.6571	0.4683	0.5462	0.8763	0.6903	0.6078
	1	0.7234	0.8842	0.7956	0.7686		
	2	0.6245	0.5914	0.6068	0.8617		
	3	0.6376	0.3888	0.4823	0.8741		
2	0	0.6653	0.4725	0.5519	0.8781	0.6937	**0.6124**
	1	0.7244	0.8859	0.7970	0.7700		
	2	0.6354	0.5969	0.6147	0.8649		
	3	0.6371	0.3940	0.4861	0.8743		
3	0	0.6691	0.4481	0.5360	0.8767	0.6852	0.5964
	1	0.7050	0.8979	0.7898	0.7564		
	2	0.6350	0.5717	0.6010	0.8630		
	3	0.6596	0.3534	0.4587	0.8743		

Table 3.6: Random Forest performance across tested configurations

For **Gradient Boosting**, we varied the number of estimators, learning rate, maximum depth, and subsampling to adjust model capacity and regularization.
For **XGBoost**, we explored depth, shrinkage (learning rate), number of trees, and L2 regularization λ to control bias–variance and improve class balance.

Config	n_estimators	learning_rate	max_depth	subsample
1	100	0.2	4	0.8
2	200	0.1	4	0.8
3	300	0.2	3	1.0

Table 3.7: Gradient Boosting hyperparameter configurations (max_features = None).

Config	n_estimators	learning_rate	max_depth	reg_lambda
1	500	0.10	3	1
2	800	0.05	5	1
3	800	0.10	5	3
4	1200	0.05	5	3
5	1200	0.10	5	5

Table 3.8: XGBoost hyperparameter configurations (subsample = 0.8).

Gradient Boosting and XGBoost were competitive but did not surpass Random Forest in our search. The best Gradient Boosting run is **Config 2** with **macro F1 = 0.5753**. For XGBoost, the best run is **Config 4** with **macro F1 = 0.5969**. In our setting, bagging-based ensembles (Random Forest) appear more robust, likely due to variance reduction and reduced sensitivity to atypical samples.

Configuration	Class	Precision	Recall	F1	Binary accuracy	Accuracy	Macro F1
1	0	0.6186	0.4690	0.5326	0.8693	0.6596	0.5662
	1	0.7068	0.8680	0.7791	0.7492		
	2	0.5883	0.5132	0.5471	0.8467		
	3	0.5271	0.3321	0.4062	0.8539		
2	0	0.6122	0.4760	0.5347	0.8681	0.6659	**0.5753**
	1	0.7066	0.8724	0.7807	0.7503		
	2	0.6068	0.5206	0.5593	0.8523		
	3	0.5689	0.3431	0.4267	0.8610		
3	0	0.6063	0.4627	0.5239	0.8664	0.6601	0.5716
	1	0.7102	0.8608	0.7782	0.7499		
	2	0.5745	0.5138	0.5419	0.8433		
	3	0.5630	0.3660	0.4423	0.8606		

Table 3.9: Gradient Boosting performance across tested configurations

Configuration	Class	Precision	Recall	F1	Binary accuracy	Accuracy	Macro F1
1	0	0.6222	0.4662	0.5320	0.8697	0.6664	0.5758
	1	0.7059	0.8750	0.7813	0.7503		
	2	0.5912	0.5187	0.5518	0.8483		
	3	0.5901	0.3505	0.4380	0.8646		
2	0	0.6340	0.4816	0.5465	0.8732	0.6799	0.5957
	1	0.7231	0.8722	0.7905	0.7644		
	2	0.6141	0.5618	0.5859	0.8571		
	3	0.5834	0.3814	0.4600	0.8650		
3	0	0.6290	0.4802	0.5439	0.8721	0.6743	0.5911
	1	0.7231	0.8628	0.7867	0.7616		
	2	0.6050	0.5551	0.5784	0.8541		
	3	0.5569	0.3859	0.4556	0.8609		
4	0	0.6239	0.4781	0.5405	0.8710	0.6797	**0.5969**
	1	0.7263	0.8687	0.7910	0.7661		
	2	0.6142	0.5606	0.5853	0.8569		
	3	0.5807	0.3969	0.4707	0.8653		
5	0	0.6266	0.4879	0.5483	0.8722	0.6766	0.5948
	1	0.7296	0.8621	0.7902	0.7668		
	2	0.6010	0.5483	0.5725	0.8523		
	3	0.5595	0.4028	0.4680	0.8618		

Table 3.10: XGBoost performance across tested configurations

3.3.4 Neural network: MLP

The MLP achieved solid performance but did not surpass the best tree-based ensemble models.
Network depth and width (hidden layer sizes) were varied, and dropout was applied to regulate model capacity and mitigate overfitting. All configurations were trained with a fixed learning rate of $1e-3$ using the Adam optimizer, a batch size of 32, and the cross-entropy loss function. Training was run for up to 200 epochs with early stopping (patience = 10) to ensure stable convergence and prevent overfitting.

Config	Hidden layers	Dropout
1	[64, 64]	0.00
2	[64, 128]	0.00
3	[32, 64, 128]	0.00
4	[64, 64]	0.10
5	[64, 128]	0.10
6	[32, 64, 128]	0.05

Table 3.11: MLP hyperparameter configurations.

The strongest configuration is **MLP Config 6** with **macro F1 = 0.5867** and **Accuracy = 0.6761**. Compared to Gradient Boosting and XGBoost, MLP performance is competitive, but the best Random Forest run still leads. In our experiments, increasing model capacity and tuning regularization improved macro F1 up to a point, after which gains saturated.

Configuration	Class	Precision	Recall	F1	Binary accuracy	Accuracy	Macro F1
1	0	0.6000	0.4529	0.5150	0.8644	0.6600	0.5663
	1	0.7183	0.8560	0.7809	0.7554		
	2	0.5672	0.5754	0.5705	0.8434		
	3	0.5512	0.3180	0.3990	0.8567		
2	0	0.6025	0.4480	0.5126	0.8647	0.6606	0.5686
	1	0.7165	0.8580	0.7807	0.7544		
	2	0.5813	0.5618	0.5698	0.8474		
	3	0.5347	0.3365	0.4112	0.8546		
3	0	0.6193	0.4704	0.5326	0.8692	0.6717	0.5847
	1	0.7361	0.8556	0.7911	0.7699		
	2	0.5804	0.5858	0.5822	0.8486		
	3	0.5351	0.3660	0.4329	0.8557		
4	0	0.6134	0.4550	0.5206	0.8669	0.6678	0.5765
	1	0.7240	0.8578	0.7850	0.7606		
	2	0.5885	0.6061	0.5955	0.8517		
	3	0.5458	0.3247	0.4049	0.8564		
5	0	0.6379	0.4662	0.5376	0.8727	0.6742	0.5850
	1	0.7227	0.8687	0.7886	0.7628		
	2	0.6051	0.5883	0.5941	0.8549		
	3	0.5518	0.3402	0.4197	0.8581		
6	0	0.6269	0.4620	0.5311	0.8702	0.6761	**0.5867**
	1	0.7272	0.8691	0.7917	0.7670		
	2	0.5981	0.5975	0.5973	0.8546		
	3	0.5660	0.3446	0.4265	0.8604		

Table 3.12: MLP performance across tested configurations

3.4 What improved performance and what did not

Several clear patterns emerged:

- **Moving from linear to non-linear models was essential.** Logistic Regression achieved macro F1 ≈ 0.25, while non-linear models (SVM and tree ensembles) improved substantially.
- **Tree ensembles provided the best robustness and class balance.** Random Forest achieved the highest macro F1 (**0.6124**), with XGBoost (**0.5969**) and MLP (**0.5867**) close behind.
- **Hyperparameter sweeps showed diminishing returns.** For boosting/MLP, improvements were incremental and saturated quickly within the tested ranges.
- **Class imbalance in difficulty remained visible.** Across most models, class 1 consistently achieved high recall, while class 3 tended to have notably lower recall/F1, suggesting overlap with other classes and/or higher sensitivity to atypical points.

3.5 Best model and confusion matrix

Based on macro F1, our best supervised model prior to explicit outlier filtering is **Random Forest (Config 2)** with **macro F1 = 0.6124**.
For the final pipeline, we apply GMM-based outlier filtering and then train a **Random Forest** classifier on the resulting inlier-only dataset; we therefore report the confusion matrix of this final Random Forest model on a held-out fold (not used for training).

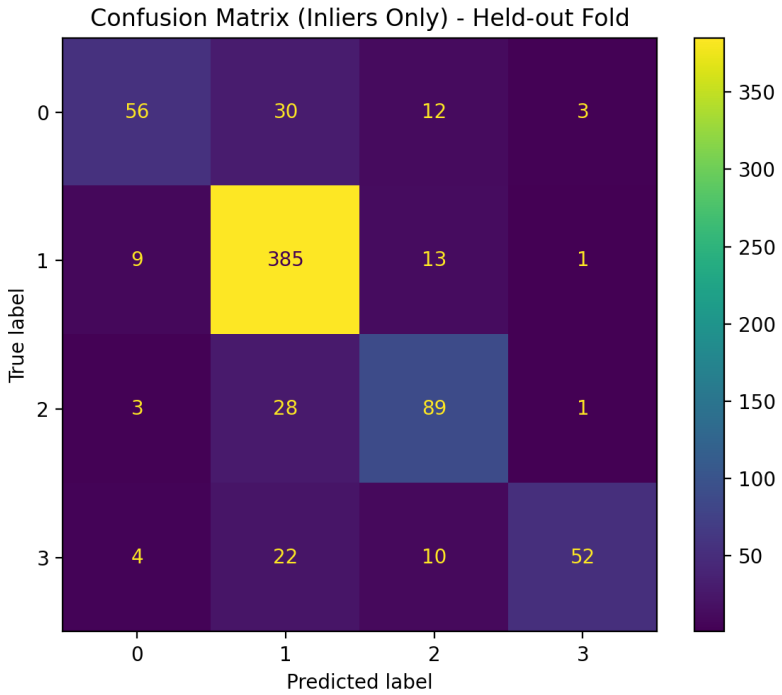

Figure 3.1: Confusion matrix of the final Random Forest model on a held-out inlier-only validation fold

3.6 If starting again from scratch

If we were to restart the project, we would:

- Prioritize **robust modeling** earlier (e.g., tree ensembles as baseline instead of linear models), given the apparent non-linear structure.
- Integrate **outlier-aware validation** sooner (e.g., evaluating performance after filtering likely outliers) to better match the real deployment setting and the leaderboard scoring.
- Perform a more systematic hyperparameter search around the top-performing families (Random Forest / XGBoost / MLP), especially focusing on improving the hardest class (typically class 3).

4 Outlier Detection

To identify outliers in the dataset, three unsupervised detection methods were implemented - Gaussian Mixture Model (GMM), Isolation Forest, and Local Outlier Factor (LOF). Each model was trained on inlier data D, and a separate outlier dataset D_{out} was used to calibrate a decision threshold.

4.1 Gaussian Mixture Model

A Gaussian Mixture Model was fitted to approximate the underlying data distribution $p_0(\mathbf{x})$. The optimal number of components was selected using the Bayesian Information Criterion (BIC). The model was trained on scaled inlier data D_{out}, and log-likelihood scores were computed for the outlier dataset D using `score_samples()` method. The threshold τ was set as the maximum log-likelihood among outliers, ensuring that points with $p(x) > \tau$ were classified as inliers, while those with $p(x) \leq \tau$ were flagged as outliers.

4.2 Local Outlier Factor

The Local Outlier Factor method was applied to identify outliers by comparing the local density of a point to that of its neighbors. An observation is considered an outlier if its local density is significantly lower than that of surrounding points. The model was trained on inlier data D_{out}, and the number of neighbors was empirically determined based on evaluation of the dataset. To calibrate the threshold τ, LOF scores were computed for the outlier dataset D using `score_samples()`. Since more negative scores indicate stronger anomaly evidence, the threshold τ was set to the highest LOF score among the outliers. This ensured that all points with scores greater than this threshold were classified as inliers, while points with scores equal to or below the threshold were flagged as outliers.

4.3 Isolation Forest

An Isolation Forest was implemented to detect outliers by randomly partitioning the feature space and measuring the average path length required to isolate each point. Points that can be isolated with fewer splits are more likely to be classified as outliers. The model was trained on inlier data D_{Out}, with the number of estimators to fixed values empirically determined from dataset evaluation. To calibrate the threshold τ, anomaly scores for inliers and outliers were computed using `score_samples()`. Candidate thresholds were taken as quantiles of the combined score distribution and evaluated by recall on outliers and false positive rate on inliers. The threshold that maximized the objective function, which prioritized recall while penalizing deviations from the target outlier rate, was selected as the final decision boundary.

Finally, after the threshold calibration, each outlier detection model was used to filter the original dataset by removing points classified as outliers. The resulting inlier-only datasets were then used to retrain the best-performing Random Forest classifier from Chapter 3. The classification results obtained after filtering are summarized in Table 4.1.

Method	Class	Precision	Recall	F1	Binary accuracy	Accuracy	Macro F1
GMM	0	0.8025	0.6190	0.6969	0.9248	0.8058	0.7366
	1	0.8275	0.9318	0.8764	0.8507		
	2	0.7373	0.7316	0.7339	0.9106		
	3	0.7878	0.5386	0.6392	0.9254		
LOF	0	0.8472	0.5117	0.6258	0.9339	0.7636	0.6715
	1	0.7931	0.9067	0.8457	0.8067		
	2	0.6715	0.6612	0.6613	0.8793		
	3	0.7297	0.4654	0.5533	0.9073		
Isolation Forest	0	0.7583	0.5436	0.6325	0.9073	0.7455	0.6085
	1	0.7762	0.9249	0.8440	0.7981		
	2	0.6240	0.5998	0.6106	0.8797		
	3	0.6468	0.2390	0.3471	0.9060		

Table 4.1: Comparison of outlier detection methods

The results clearly show that the choice of outlier detection method has a substantial impact on classifier performance. When trained on data filtered with GMM, the classifier achieves significant improvement and the strongest overall performance, reflected in the highest accuracy and macro F1 score. LOF provides moderate gains, offering a noticeable but smaller boost compared to GMM. In contrast, Isolation Forest yields the weakest classifier performance, suggesting that its filtering either removed too much informative data or failed to effectively identify outliers. Overall, GMM stands out as the most effective outlier filtering strategy for enhancing the classifier's predictive quality.

Appendix

Machine Learning for Artificial Intelligence Engineering, Winter Term 2025
Homework 2
Course Project

Maximilian Baronig

Tutor:	Nicolas Pfob, nicolas.pfob@student.tugraz.at
Points to achieve:	35 Points
Deadline:	23.01.2026 23:59 (strict, no late submissions allowed)
Hand-in procedure:	Submit **a report (PDF)**, the predictions `submission_leaderboard_GROUPNAME.csv`, `submission_final_GROUPNAME.csv`, and your project code/assets as `Code.zip`
Plagiarism:	If detected, 0 points on the entire assignment sheet for all parties involved. If this happens twice, we will grade the group with "Ungültig aufgrund von Täuschung"
Course info:	TeachCenter, https://tc.tugraz.at/main/course/view.php?id=6273

Contents

1 Problem Description **2**

2 Exploratory Data Analysis [6 points] **3**

3 Baseline Model [3 points] **3**

4 Model Experimentation & Validation [19 points] **3**

5 Outlier Detection [7 points] **4**

6 Leaderboard Predictions [Bonus Points & Prizes] **4**

General remarks

Your submission will be graded based on:

- Correctness (Is your code doing what it should be doing? Is your derivation correct?)
- The depth of your interpretations (Usually, only a couple of lines are needed.)
- The quality of your plots (Is everything clearly readable/interpretable? Are axes labeled? …)
- Your submission must run with Python 3.11.5. In your submitted `Code.zip`, there should be a `requirements.txt` with all packages one needs to install to run your code (with their versions). You are free to use **any** third-party packages in this project.

The usage of LLMs and LLM-based coding assistants is **explicitly allowed** for this assignment. Feel free to use them both for coding, and for coming up with ideas/strategies to tackle the issues you face.

1 Problem Description

You work in a company that produces sensors that measure several physical quantities at the production sites of your customers. Your company has developed a sensor that can detect if a certain discrete event $y \in \{0, 1, 2, 3\}$ occurs at a production facility of your customer. Moreover, the sensor records auxiliary information about the environment (scalar values, collected in $\mathbf{x} \in \mathbb{R}^d$, $d = 12$) when it reports the event y. However, the parts of the sensor that "directly" measure y are extremely expensive, whereas the parts that measure the auxiliary information $\mathbf{x}$ are *much* cheaper. The company also assumes that there is some relationship between $\mathbf{x}$ and y, which you are tasked to find. If there was a model f_θ that predicts y from $\mathbf{x}$ well, your company could omit the expensive parts of the sensor, just record $\mathbf{x}$ and predict y using f_θ.

Data Collection. To collect data, the company intern is tasked to mount one sensor at each of the three different production sites at your customer's facility (i.e., three sensors in total). Since there is no time dependency in any of the measurements, we can assume that the recorded pairs $(\mathbf{x}, y)$ are i.i.d. samples of some true, unknown distribution $p^*(\mathbf{x}, y)$. Unfortunately, the intern forgot to note down which recordings came from which sensor. Thus, you are given a dataset $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^N$ where $\mathbf{x}^{(i)} \in \mathbb{R}^d$ and $y^{(i)} \in \{0, 1, 2, 3\}$ for each $i \in \{1, \ldots, N\}$, with $d = 12$ and $N = 9000$.

Outliers. Sometimes, the mounted sensors are known to malfunction randomly: In these cases, the recorded $(\mathbf{x}, y)$ pairs are highly atypical, i.e., $\mathbf{x}$ is not a sample from $p^*(\mathbf{x})$, and y is not a sample from $p^*(y \mid \mathbf{x})$. Roughly 20% of the recordings are known to be outliers. In these cases, your model should not just predict $\hat{y} = f_\theta(\mathbf{x})$, but should check if $\mathbf{x}$ looks like an outlier (if so, this could be displayed to the user instead of blindly showing a prediction $\hat{y}$). To this end, the company also supplies you with a (small) set of auxiliary recordings $\mathcal{D}_{\text{out}} = \{\mathbf{x}^{(i)}\}_{i=1}^O$, with $O = 57$, which are known to be outliers (manual inspection).

Leaderboard. In this assignment, you will try to solve these problems by applying various machine learning techniques. Similar to popular online machine learning challenges, there will be a public leaderboard on TeachCenter which will evaluate the performance of your models on a hidden test set which you will have no access to. The structure is based on the workflow of real-world machine learning projects and encourages you to be creative about your approaches. We will focus on experimentation, evaluation and interpretability of different machine learning methods.

Supplied Data Files. Regarding the data, you will receive the following files:

- `D.csv`, containing the data in $\mathcal{D}$
- `D_out.csv`, containing the data in $\mathcal{D}_{\text{out}}$
- `D_test_leaderboard.csv`, containing test points $\{\mathbf{x}^{(i)}\}_{i=1}^{2943}$ you will need to classify to get a *public* leaderboard score prior to the deadline
- `D_test_final.csv`, containing test points $\{\mathbf{x}^{(i)}\}_{i=1}^{3000}$ you will need to classify to get a *final* score on the leaderboard *after* the deadline (see Section 6)

2 Exploratory Data Analysis [6 points]

After loading the data, your task is to perform (basic) *Exploratory Data Analysis* (EDA), e.g. distributions or correlations in the dataset. Provide **at least five** meaningful plots and include them in the report. Meaningful in this context means, that patterns, relationships, or anomalies in the data are revealed that can inform and justify modeling decisions. You can use any plotting library you like, but make sure the axes, legends and titles of the plots are informative and meaningful. Some ideas: Dimensionality reduction, clustering, plotting distribution of class labels, correlation between features, etc.

Include in your report:

- Five to ten meaningful and readable plots, as outlined above. [4 points]
- A single paragraph, providing a brief interpretation on all included plots and how they might influence decision making during the experiments. [2 points]

3 Baseline Model [3 points]

You do not want to just evaluate the performance of your model using the public leaderboard. Instead, carefully think about how you want to evaluate your model locally. Pick a validation strategy that you think best represents the actual performance of your models. Be sure to avoid common pitfalls, like label leakage. You will use this validation strategy throughout this project.
Next, train a simple `KNeighborsClassifier` from the `sklearn` library to get your first results. This baseline model will be the starting point of your experiments. Using your validation strategy, pick a good value for K. Think about if you want to preprocess the data. For each of the four classes, compute *Precision*, *Recall*, the F_1 score, and the *Accuracy* of your baseline model. Moreover, compute and report the unweighted average of the per-class F_1 scores (macro[1] average).

Include in your report:

- Briefly describe your validation strategy. [1 points]
- Report your choice of K. Using the baseline model, report *Precision*, *Recall*, F_1, *Accuracy* (for all classes), and the macro-averaged F_1 score. [2 points]

4 Model Experimentation & Validation [19 points]

After obtaining initial results from our baseline model and deciding on a validation strategy, your next task is to iteratively improve your approaches. Train models with **at least three different architectures** and show that you performed **at least ten total experiments** with different hyperparameter configurations, and/or feature/data engineering approaches.
Choices in models, optimizers, regularization techniques, hyperparameters, feature engineering approaches, etc. are completely up to you. You can leverage models we have discussed in this course, but feel free to experiment with models we have not covered in class: Exploration and experimentation are strongly encouraged! If you do not know where to start, we recommend exploring the following models: Logistic Regression, Support Vector Machines, Decision Trees, Random Forests, Gradient Boosted Trees (e.g. with `xgboost`, `LightGBM`), MLPs, or Gaussian Processes. Even though MLPs (neural networks) are not covered in this lecture, you are allowed to use them as one of the three architectures.

Data Preprocessing. For each model type you choose, think critically about whether data preprocessing is necessary or beneficial. Do you need to preprocess the features for your model (e.g. standardization)? Are there any additional features that you could add or can removing features improve performance?

[1] See e.g. `https://scikit-learn.org/stable/modules/generated/sklearn.metrics.f1_score.html`

Validation. Evaluation is crucial to compare the different experimental settings and models. Stick to the validation strategy you already chose carefully in Task 3 and for all experiments, report per-class F_1, Precision, Recall, Accuracy, and macro-weighted F_1. Be sure to avoid common pitfalls, like label leakage, different class proportions or wrong preprocessing on a test set. Hint: If your validation scores are close to your leaderboard scores, you are on the right path. **Important: Insufficient local validation will lead to point deductions!**

Include in your report:

- For each experiment, report per-class F_1, Precision, Recall, Accuracy, and macro-weighted F_1. A minimum of ten total experiments is required with at least three different architectures. [10 points]
- Explain briefly what decisions you have made during the experiments and why (e.g. 'we increased the weight decay because our model showed signs of overfitting'). [3.5 points]
- What changes led to performance improvements and what changes did not? Your report should give a good overview of the methods you have tried, even if they have not led to a performance improvement. [3.5 points]
- Using the model you consider best, plot a confusion matrix with data not used for training. [1 points]
- If you were to start again from scratch what would you do differently? [1 points]

5 Outlier Detection [7 points]

So far, we have focused on the supervised learning task of predicting y from $\mathbf{x}$. We will now work on detecting whether a given $\mathbf{x}$ is an *outlier*, as described in Section 1. There are many ways to perform this task, and you are free to explore any of them.
One way to tackle this problem is to learn a probabilistic model $p_\theta(\mathbf{x})$ such that it fits the data distribution well, i.e., $p_\theta(\mathbf{x}) \approx p^*(\mathbf{x})$ for all $\mathbf{x} \in \mathbb{R}^d$. Since we assume that outlier $\mathbf{x}$ are "unlikely" under p^*, we may expect that p_θ also assigns low density to this outlier. For example, you could fit a *Gaussian Mixture Model* (GMM) $p_\theta(\mathbf{x})$ using the features in our training data $\mathcal{D}$, and proceed to use this GMM to detect outliers. A simple way to get started would be to figure out a threshold $\tau > 0$ such that typically, $p(\mathbf{x}_{\text{in}}) > \tau$, and $p(\mathbf{x}_{\text{out}}) \leq \tau$, where $\mathbf{x}_{\text{in}}$ is an inlier, and $\mathbf{x}_{\text{out}}$ is an outlier. You may use $\mathcal{D}_{\text{out}}$ to determine such a threshold.

Using the outlier detection approach of your choice, filter the original dataset $\mathcal{D}$ to obtain a new dataset that excludes outliers ($\mathcal{D}_{\text{in}}$). Re-train the model you considered best on $\mathcal{D}_{\text{in}}$. Does it improve performance? Include in your report:

- Clearly explain your strategy for outlier detection. Which model did you train and how did you use it to label instances $\mathbf{x}$ as inliers and outliers? If you have tried multiple approaches, briefly describe all of them. [5.5 points]
- Report how the performance of your best model changed when training on inliers only. [1.5 points]

6 Leaderboard Predictions [Bonus Points & Prizes]

Select the model you think works best. For each test data point $\mathbf{x}$, use this model to predict (1) the class label $\hat{y} \in \{0, 1, 2, 3\}$, and `is_outlier` $\in \{0, 1\}$. Save your predictions in the correct format (see `code_skeleton.py`) on the provided test sets as the following `csv` file :

- Save predictions on `D_test_leaderboard.csv` as `submission_leaderboard_GROUPNAME.csv`, where `GROUPNAME` should be your group name that will be displayed on the leaderboard.
- Save predictions on `D_test_final.csv` as `submission_final_GROUPNAME.csv`, which will be used for the final leaderboard score (after the deadline).

Feel free to pick `GROUPNAME` to be different from your Group ID (artistic freedom).

Leaderboard Phases. There will be two leaderboad "phases":

1. **12.12. 23:59 – 23.01. 23:59**: The leaderboard will score the uploaded `submission_leaderboard_GROUPNAME.csv` file and display the results (discussed below) on the leaderboard on TeachCenter. The leaderboard will update everyday at the following times: $\{06{:}00, 09{:}00, 12{:}00, 15{:}00, 18{:}00, 21{:}00, 24{:}00\}$ and will only consider the currently uploaded submission with the correct name.

2. **24.01. 00:01**: On Saturday after the deadline, we will score the predictions in `submission_final_GROUPNAME.csv` (which was uploaded before the deadline). The score we compute here is the **final leaderboard score** and will determine the **winners** of this competition. The top 10 places will receive **bonus points**:

   - 1 bonus point for rank 10
   - 2 bonus points for rank 9
   - ...
   - 8 bonus points for rank 3
   - 9 bonus points for rank 2
   - 10 bonus points for rank 1

Leaderboard Scoring. Note that the test data for the public leaderboard again contains *inlier* and *outlier* points (the same is true for the hidden final test dataset). However, we only consider predictions of your model for *inlier* data points: We compute the *unweighted* (macro) average of the per-class F_1 scores, denoted as F_1^{macro}, since we care equally much about predictions for each class. In the leaderboard, we display this as $\texttt{ClassScore} = 100 \cdot F_1^{\text{macro}}$. Furthermore, we compute the F_1 score of the binary predictions `is_outlier` on *all* the test data, shown as $\texttt{OutlierScore} = 100 \cdot F_1$ in the leaderboard. Finally, the leaderboard score `Score` is given by

$$\texttt{Score} := 0.8 \cdot \texttt{ClassScore} + 0.2 \cdot \texttt{OutlierScore}$$

We will provide a tutors baseline and a random baseline as an orientation. **Important: Stick to the format of the sample submission, any deviations will lead to an error during scoring!**